Large-Scale JSON Tree Retrieval: A Single-Host Storage Microbenchmark

Junyao Dong

Abstract. Large JSON tree stores need fast node lookup, child expansion, subtree retrieval, and entity fetch. We benchmark their storage-level equivalents on one ten-million-node synthetic tree with warm caches and a local client. MongoDB's short reads have higher fixed request and query setup than PostgreSQL, while subtree retrieval also pays more per-key cursor work and covered-row handoff. Against sequential individual queries, coalescing three already-known inputs reduces MongoDB group-completion P50 by 2.66×, 2.27×, and 2.62× for node, child, and entity reads; at 64 inputs the corresponding ratios are 24.1×, 6.81×, and 25.1×. With at most 16 client workers, a threaded control still leaves coalescing 1.56–2.28× faster at three inputs across two grouping seeds. Cursor batch size has no projection-independent optimum: a one-million-row request gives a modest ID-only improvement but not a covered-Metadata improvement. A frozen 256-row DFS bucket layout improves large-subtree P50/P95 by 1.57×/1.72× on the full root-ID-disjoint split; after removing all ancestor-correlated roots, the ratios remain 1.56×/1.67×. Small subtrees regress. A candidate direction is to coalesce naturally grouped short reads and investigate routing only large subtrees through buckets, but the MongoDB interventions remain storage-harness experiments rather than integrated ConDB paths. In a separate MongoDB master-snapshot source experiment, letting a direct `CountStage`→`CountScan` pair skip an unread working-set member reduces benchmark-thread user-space retired instructions against a pinned upstream base carrying the same benchmark harness by 4.600%, 2.055% and 3.680% on scalar, multikey and compound-wildcard indexed counts, favourably in all 30 blocks — about 117 instructions per counted document. It fires wherever `CountStage`'s child is a bare `CountScan`, which the planner guarantees for every classic `COUNT_SCAN` plan; counts that plan to `FETCH` or a collection scan are untouched. Every plan measured is forced by an index hint. That campaign's pre-registered adoption gate nevertheless did not pass, because both of its control workloads fell outside their bands; no CPU-time or wall-time result is claimed from it, and it is not a 7.0.34, ConDB, or production result. The SQLite-backed Beam path now coalesces its per-turn child and entity reads. With the public defaults (`beam_size=3`, `select_k=1`), a file-backed control returns one leaf and its content, reduces batchable reads from 37 to five and all SQLite `SELECT`s from 39 to seven, and improves P50 by 1.177–1.192×. At 16 clients, median throughput improves from 237.3 to 272.2 queries/s, although absolute throughput falls as client count rises. Additional trees and mutable-tree support remain to be tested.

1 OPERATIONS AND LAYOUTS

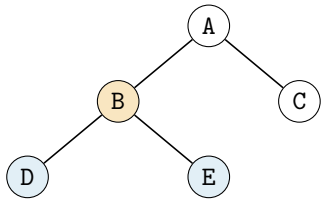
We benchmark fully materialized reads on a large tree. MongoDB and PostgreSQL are compared under Materialized Path; PostgreSQL and SQLite are also measured under Order/Size. Inputs, ordering, returned fields, and logical results are identical. Entity fetch is layout-independent because the two hierarchy layouts in each SQL engine call the same shared Text query.

Table 1: The four storage operations measured in every configuration.

Operation	Returned data	Retrieval role
`get_node`	One node's Structure and Metadata	Read or navigate to one node
`get_children`	Ordered direct-child Metadata	Choose the next branch
`get_subtree`	Ordered descendant Metadata	Render a tree view or retrieval block
`get_entity`	One selected node's Text	Read final content

We compare two established hierarchy representations. Materialized Path stores each node's root-to-node path as a string.[1] Order/Size labeling assigns each node a preorder rank and subtree size.[2] Our implementation stores the equivalent exclusive bound `end = pre + size`; a node v is a descendant of u when $\text{pre}(u) < \text{pre}(v) < \text{end}(u)$. Both representations keep Structure and Metadata together. MongoDB and PostgreSQL use dedicated Text stores; the two SQLite layouts share the original source table for entity fetch.

[1] MongoDB, "Model Tree Structures with Materialized Paths."
[2] Q. Li and B. Moon, "Indexing and Querying XML Data for Regular Path Expressions," VLDB 2001.



(a) Logical tree

node	path
A	/A
B	/A/B
D	/A/B/D
E	/A/B/E
C	/A/C

`/A/B/ <= path < /A/B0`

(b) Materialized path

node	pre	end
A	1	6
B	2	5
D	3	4
E	4	5
C	5	6

`2 < pre < 5`

(c) Order/Size labeling

Figure 1: The same logical tree and its two hierarchy encodings for `get_subtree(B)`. Orange marks the query root; blue marks the returned descendants. In both layouts the descendants occupy one contiguous index range. Common node fields and entity storage are omitted.

Table 2: Five measured engine/layout configurations.

Engine	Materialized Path	Order/Size
MongoDB	Nodes + Text collections; path string	Not tested
PostgreSQL	Nodes + Text tables; path string	Nodes + Text tables; `pre,end`
SQLite	Path nodes + shared source table	Order/Size nodes + shared source table

2 METHOD

2.1 Experimental setup

All runs use a dual-socket Intel Xeon Gold 6418H host with 48 physical cores, 96 hardware threads, and 1 TiB RAM. Except for the separate source experiment in Section 5, all ConDB storage-harness and root-cause measurements use MongoDB 7.0.34; that experiment uses master snapshot `5d3b36cf3871` and neither reruns nor replaces the 7.0.34 measurements. PostgreSQL 16.14 runs as a localhost service; the base SQLite storage experiments use SQLite 3.50.4 in process. The cross-engine implementation controls in Section 6.5 were run separately with system Python 3.10.12 and SQLite 3.37.2; their raw files record the exact environment. Database and operating-system caches are not flushed.

The base layout and root-cause timings are sequential and single-client. The short-read section adds one 16-worker-capped group-completion control, and Section 6.5 separately measures throughput at one, four, and 16 clients. Mixed reads/writes, longer-duration load, and configurations above 16 clients are not evaluated.

The benchmark uses point and parent indexes for node and child reads, Materialized Path or Order/Size indexes for subtree reads, and a point index for Text. The PostgreSQL and SQLite subtree indexes cover the returned Metadata. MongoDB's subtree index also carries `node_id`, title, and summary, so its range scan does not require document fetch.

Each configuration is warmed up, measured in cyclically interleaved order for three repeats, and summarized from the per-input median. SQLite is an in-process reference and is not used for server-to-server ratios. Optimization experiments state their own repeat counts and also summarize per-input medians. Base layout comparisons do not report confidence intervals, so sub-millisecond differences there are descriptive. The cursor experiment reports descriptive root-bootstrap intervals for paired medians; they do not model hierarchy/branch clustering. Unless stated otherwise, P95 is therefore a workload percentile across inputs after per-input repetition, not an open-loop service-latency tail over repeated arrivals of one request.

2.2 Dataset and workload

The benchmark uses the same synthetic ten-million-node tree in all five configurations. It has templated random content, fanout 6–14, and maximum depth eight. `get_node` and `get_entity` use the first 500 leaf IDs ordered by `node_id`; `get_children` and `get_subtree` use the same 200 deterministic roots. The point and entity results characterize these fixed inputs; text sizes are not stratified. The subtree arm exercises the expensive descendant-range core: it excludes the root, has no depth limit, returns `(node_id,title,summary)`, and averages 36,456.7 rows. This storage microbenchmark is not the public ConDB API, which includes the root and applies a caller-supplied depth limit. All four measured operations are storage-level experimental equivalents. The MongoDB layouts and DFS buckets are not implemented in the current SQLite-backed public API. Short-read coalescing for child and entity reads is implemented inside the Beam retrieval path, but the public navigation methods remain scalar. Method names, projected record shapes, and path/node ordering also differ from the public facade, which exposes `get_content` and slot-ordered children.

Each configuration contains one sealed logical tree; multi-tree execution is outside this experiment. Every result is fully materialized. All 21,000 timed results match in row count and SHA-256 fingerprint.

2.3 Root-cause controls

For `get_node` and `get_entity`, hit/miss pairs show how much work remains when no row is returned. PostgreSQL runs without statement preparation for the main comparison; prepared execution is included as a sensitivity check. The entity test also varies Text from zero to 256 KiB. The benchmark's default MongoDB query is left unhinted and uses `IDHACK`.

For `get_children`, a synthetic tree varies fanout from zero to 128. Covered ID, covered Metadata, and noncovered plans expose fixed setup, covered row processing, and the document-access path.

Each short read is then repeated as one hot query shape while CPU cycles are sampled from the running MongoDB binary. Its `gitVersion` matches the `GitOrigin-RevId` in the official `r7.0.34` source tag. The sampled functions identify execution paths; their inclusive call-graph shares are not additive. A command-boundary control separately measures the Python driver wrapper.

For `get_subtree`, a scalar count removes BSON row output and client materialization. Long-range CPU counters measure instructions and cycles per key. PostgreSQL `EXPLAIN` verifies the plan but is not used as a latency measurement.

3 LATENCY OF ALL FOUR OPERATIONS

MongoDB and PostgreSQL form the server-to-server comparison. SQLite runs in process and is shown only as an embedded layout reference; its absolute latency is not ranked against the two server engines.

Table 3: P50/P95 latency in milliseconds. Each cell is computed from per-input medians over three interleaved repeats. The MongoDB and PostgreSQL Path columns form the server-to-server comparison; the remaining SQL columns show within-engine layout effects. Lower is better.

	MongoDB	PostgreSQL		SQLite	
Operation	Path	Path	Order/Size	Path	Order/Size
`get_node`	0.196 / 0.220	0.092 / 0.107	0.082 / 0.093	0.012 / 0.016	0.008 / 0.010
`get_children`	0.252 / 0.292	0.100 / 0.113	0.099 / 0.117	0.046 / 0.062	0.067 / 0.092
`get_subtree`	18.1 / 208.1	13.6 / 164.9	12.5 / 156.8	9.94 / 124.3	9.84 / 124.9
`get_entity`	0.142 / 0.166	0.074 / 0.082	0.070 / 0.077	0.008 / 0.010	0.005 / 0.006

At P50, the MongoDB–PostgreSQL gaps are 0.104 ms for node lookup, 0.152 ms for child expansion, 4.5 ms for subtree retrieval, and 0.068 ms for entity fetch. The subtree gap reaches 43.2 ms at P95. The short-read gaps are small in absolute time; subtree retrieval is the only operation where scan and row-emission cost materially changes latency.

Hierarchy representation matters primarily for subtree retrieval. PostgreSQL Order/Size reduces subtree P50 from 13.6 to 12.5 ms; SQLite changes by only 0.10 ms. Node and child differences are at most 0.021 ms and are descriptive at this scale. The two entity columns in each SQL engine repeat one shared Text query, so their tiny differences are timing noise, not a hierarchy-layout effect. The integer-bound subtree result is consistent with lower key-processing cost, but the independently built layouts do not isolate that factor alone.

4 MONGODB EXCESS LATENCY

The controls in Table 4 distinguish fixed request setup from row-dependent scan and materialization work.

Table 4: Root-cause controls for the four operations. Child slopes compare MongoDB with unprepared PostgreSQL in μs per returned child. The covered and noncovered child arms also differ in filter, projection, and index shape, so their contrast localizes an execution bundle rather than a pure fetch cost.

Operation	Quantitative control	What the control establishes
`get_node`	MongoDB hit minus miss: 12.5 μs P50; command boundary: 176.5 of 249.8 μs inside the command	Row fetch is a small increment; most observed wall time lies within the command path.
`get_children`	Covered intercept/slope: 263/1.64 vs. 117/0.97; noncovered: 264/2.16 vs. 109/0.99	MongoDB shows both a larger fixed component and a steeper returned-row component; the noncovered bundle is steeper again.
`get_subtree`	Scalar count: 7.44 vs. 4.34 ms; 597 vs. 259 cycles/key	The gap survives removal of BSON row output and tracks higher per-key scan work.
`get_entity`	0 vs. 1 KiB text: 0.228 vs. 0.224 ms in MongoDB; 0.096 vs. 0.097 ms in PostgreSQL	At the benchmark's 1 KiB payload, text transfer is below run-scale variation; fixed lookup work dominates.

4.1 Node lookup

For `get_node`, the hit–miss increment is only 12.5 μs, while 176.5 of 249.8 μs lies inside the MongoDB command. Source-localization samples show `find` parsing, canonicalization, collection acquisition, `QueryPlanner::plan`, and

stage-tree construction. Document fetch is a small part of this point lookup; fixed command and planning work dominates.

4.2 Child expansion

Both engines use a parent index followed by base-record lookup for `get_children`. MongoDB has a higher fitted intercept and a steeper covered-row slope; the noncovered arm is steeper again. Samples show `IndexScan`, KeyString decoding, projection, and response append for covered rows. The noncovered path additionally includes `FetchStage`, `WorkingSetCommon::fetch`, and record-store seek. Because the covered and noncovered arms also use different filters and indexes, their difference is an access-path cost rather than a pure fetch measurement.

4.3 Subtree retrieval

For `get_subtree`, MongoDB's overhead has two components: per-key cursor and executor work during the range scan, and per-row handoff during covered result production.

Table 5: Subtree scan cost. Mean latency uses 200 ranges on the ten-million-node tree. CPU counters use a fixed 500,000-key long-string range.

Metric	MongoDB	PostgreSQL	Ratio
Mean scalar-count latency (ms)	7.44	4.34	1.71×
Instructions per key	2,438	1,121	2.17×
CPU cycles per key	597	259	2.30×

Samples place the work in WiredTiger cursor advance and key extraction, the storage adapter, and classic executor handoff, with `__wt_btcur_next` the largest single hotspot. That profile describes this scalar count specifically and must not be read as characterizing subtree retrieval in general: a scalar count runs `COUNT_SCAN`, which requests index keys without materializing them, so it performs no per-row BSON work. On the document-returning covered path the profile inverts. Aggregating the same samples by symbol for a covered child scan, key decoding, BSON construction and the `Document` wrapper together account for roughly three times the cycles attributed to WiredTiger, because each returned row is decoded into one BSON object, copied into a second, wrapped as a `Document`, unwrapped, and copied once more into the reply. The per-key finding above therefore motivates the count experiment in Section 5, not the subtree path.

4.4 Entity fetch

For `get_entity`, samples localize the fixed lookup cost to the `IDHACK` command path, including request parsing, canonicalization, collection acquisition, executor initialization, `IDHackStage`, and projection construction. The payload control shows no measurable P50 change at 1 KiB. At 256 KiB, latency rises to 0.391 ms in MongoDB and 0.619 ms in PostgreSQL. Fixed lookup work dominates only at the 1 KiB size used by the main workload.

The short-read arms pin an explicit index hint on both the baseline and the candidate. That keeps the comparison like-for-like, which is what the coalescing result depends on, but it also means the baseline is not the fastest path the server offers: a hint suppresses the point-query fast paths, costing about 19 µs (roughly 9%) per lookup against an unhinted `IDHACK` on 7.0.34. On a current master snapshot the same hint would suppress the newer `EXPRESS` fast path as well. The coalescing ratios reported here are therefore ratios between two hinted arms; the absolute baseline latency would be modestly lower without the hint, and a deployment that can express its lookup as a single-field or `_id` equality without a hint already receives part of this benefit from the server.

5 SEPARATE MONGODB MASTER-SNAPSHOT SOURCE EXPERIMENT

The per-key root-cause result above motivated one narrow source change on the pinned MongoDB master snapshot `0561c098b99a`. A classic `CountStage` reads only its child's stage state and immediately frees any working-set member it receives, so when its direct child is a `CountScan` that member is built and torn down for nothing on every key the scan advances on. The candidate lets such a child skip producing it.

5.1 What the change is, and what it is not

The candidate is `90814b83d3e5` on `carsontung666/mongo`. It adds one boolean to `CountScan`, one branch in `doWork()`, a private setter that only a direct `CountStage` parent can reach, and the `stageType()` check in `CountStage`'s constructor that is the setter's only call site. The setter is private because `WorkingSet::get()` is checked only in debug builds, so a public mutator that switched the output contract would be an out-of-bounds read in release builds awaiting its second caller. Deduplication, memory accounting and its limit, write-conflict and yield handling, and save/restore all run before the skipped step, and `CommonStats` and `CountScanStats` are unchanged.

The optimization applies when `CountStage`'s child is a bare `CountScan`. That is not the narrow case it first appears to be: `QueryPlanner::plan` returns a count scan as the single solution as soon as one candidate is available, so a `COUNT_SCAN` plan is never multi-planned and never plan-cached, and the opt-in covers the whole classic `COUNT_SCAN` fast path. What it does not cover is counts that plan to `COUNT→FETCH→IXSCAN` or to a collection scan, where the child is a different stage.

An earlier implementation of the same idea, `4109dcc31ff6`, was rejected in independent review: it added a template to `PlanStage`, the base class of every classic stage, for one caller; a test-only `friend` in a production header; a second entry point on `CountScan`; and an unrelated devirtualization. It is retained here as a measured arm, because the question of what that additional machinery bought is answerable rather than arguable.

5.2 Three-arm protocol

Three arms are reconstructed from the same base commit in one worktree and overlaid with a byte-identical benchmark harness, so that exactly six production files may differ:

Table 6: Arms. Arm A is not an unmodified upstream binary: it carries the same harness as the others, including an evidence-only point-query control that is not part of the pull request.

Arm	Source	Role
A	`0561c098b99a`	pinned upstream base
B	`4109dcc31ff6`	the rejected heavyweight implementation
C	`90814b83d3e5`	the minimal candidate

Five workloads run per block: three indexed-count endpoints (scalar, multikey, and a non-multikey compound wildcard index), a hinted unique-field point query as a negative control, and a count whose plan is `COUNT→FETCH→IXSCAN`. The last is the only workload where a regression could hide. The optimization cannot fire on it or on the point query, but only the fetching count drives on the order of one stage iteration per document through stages the change does not touch, which is what the rejected arm's base-class refactor would perturb.

The campaign is 30 blocks × 5 workloads × 3 arms = 450 fresh processes, each contributing five technical repetitions that are averaged within the process. Blocks are stratified over the six arm-order permutations crossed with five workload rotations, every combination appearing exactly once, and execute in a seeded permuted order so that no stratum sits at a fixed period in time. Every process is pinned to one CPU under the performance governor, with the governor re-checked before each process and the exact busy fraction of that CPU and its SMT sibling recorded across each process.

Adoption reads one interval family: a stratified log-ratio Welch-t interval over the 30 block ratios, Bonferroni-adjusted across the three count endpoints. A percentile bootstrap is retained only as a sensitivity output and is structurally excluded from every gate. Retired instructions are primary, CPU time secondary, wall time auxiliary.

C versus B is registered as a *noninferiority* comparison and is reported as a complexity trade-off rather than as an adoption gate. B does everything C does and more, so it is expected to be marginally faster, and requiring C to beat it would be a test designed to fail; the question is whether the difference is worth the maintenance cost. CPU time is gated on C/A only: at this block count it resolves to roughly ±1.5% as pre-registered, and 1.0–1.3% in the event, which cannot separate two implementations expected to differ by about one percent, so the C/B CPU interval is left ungated rather than tested against a bound the instrument cannot reach. In the event the point-query control showed that CPU time carries a binary-level offset on this host, so no CPU-time result is claimed at all; see below.

5.3 Results

The pre-registered adoption gate did not pass. Both control workloads fell outside their pre-registered bands, for three distinct reasons: one band was mis-specified, one binary-level difference is real but cannot be caused by the change, and one arm genuinely does intrude on a path the optimization never touches — the rejected implementation, not the candidate — which means the C/B leg of that control could never have passed as specified.

The count-endpoint instruction results below are nevertheless reported. The protocol registers no rule for what may be published when a control fails, so that decision is itself post-hoc, and the reasons for it are given here so a reader can weigh them: these are the pre-registered primary metric, the widest of their three adjusted intervals spans better than one part in 10^4, they are favourable in every one of the 30 blocks, and neither control failure touches them.

Table 7: Candidate versus pinned upstream base, retired benchmark-thread instructions. Bonferroni-adjusted 98.333% two-sided intervals over 30 block ratios. Every block favours the candidate on every endpoint.

Endpoint	Ratio	Reduction	Adjusted CI	Blocks
Scalar, 400,000 docs / 400,000 keys	0.954005	4.600%	[4.598%, 4.601%]	30/30
Multikey, 200,000 docs / 400,000 keys	0.979446	2.055%	[2.051%, 2.060%]	30/30
Wildcard, 200,000 docs / 200,000 keys	0.963198	3.680%	[3.678%, 3.683%]	30/30

Expressed as work removed rather than as a ratio, the saving is **118.0, 116.1 and 117.1 retired instructions per counted document**. The multikey workload is the one that discriminates: it advances over 400,000 index keys while counting 200,000 documents, and its saving lands at 116.1 instructions per *counted document* rather than per key. On the other two endpoints keys and documents coincide, so they cannot separate the two normalisations and are consistency checks rather than independent evidence. That single discriminating result is what avoiding one working-set allocation, member initialisation and free per counted document predicts, and it lets a reader compute the effect at any scan size instead of transplanting a percentage.

5.3.1 Why the gate did not pass. The point-query control is unchanged in instructions, as required (0.9997, 95% CI [0.99884, 1.00055]), but its CPU-time interval was not wholly inside its band: against a $[0.97, 1.03]$ band the interval is $[0.967321, 0.987021]$, so the point estimate of 0.977 sits inside and the lower bound falls below. The offset is roughly 2.3%, appears in all six arm-order strata, survives minimum-based aggregation, and decomposes onto the arms (22920, 22767 and 22396 ns for A, B and C) rather than onto execution position (22721, 22657 and 22706 ns for first, second and third within a block). The change cannot execute on that plan, so this is a property of the binaries and not of the optimization — most likely code layout, though this campaign counts only instructions and elapsed time and so cannot separate clock from work.

The consequence is that **no CPU-time or wall-time claim is made anywhere in this experiment**, including for the count endpoints. This is a decision to discard a result the protocol permitted, not a result the protocol denied: the pre-registered CPU non-regression gate *passed* for C/A on all three endpoints and marked a CPU speed-up claim eligible, with point estimates of 0.956, 0.962 and 0.935. Those would have read as a 4–6% CPU reduction. The same instrument reports 0.977 on the point query and 0.991 on the `FETCH`-based count, two plans on which the candidate's instruction count is unchanged, so a few percent of apparent CPU improvement is available to this binary on code the change never executes. The count-endpoint CPU numbers cannot be separated from that, and are therefore not claimed.

The un-optimized control failed for two distinct reasons. Its band was $\pm 0.2\%$, applied to a between-process ratio whose per-block standard deviation turned out to be 1.57%; at that dispersion a 95% interval reaches a 0.2% half-width at roughly 240 blocks, so as specified it was never attainable at 30. The protocol records a written justification for the 1% noninferiority margin but none for either control band, so the band is best described as having been set by analogy with repetition-level reproducibility rather than from any recorded analysis of between-process dispersion.

The dispersion itself is not diffuse noise but a per-process two-state latch: each of the 90 processes lies wholly in one of two states 2.32% apart, 56 in the lower and 34 in the upper, and no process straddles them at repetition level. Within a state *and* within an arm the process-to-process coefficient of variation is at most 0.0025%; pooling arms within a state leaves a 0.26% spread, which is the rejected arm's offset. Conditioning on the state, the candidate is indistinguishable from the base on this workload (0.999990 over the 10 blocks where both landed in the lower state, 1.000000 over the 7 in the upper; standard deviations 2.8×10^{-5} and 2.2×10^{-5}), which answers the control's question even though its gate could not be evaluated as written.

That conditioning is **post-hoc**. It is not in `analyze.py`, which was hashed into the pre-registration and knows nothing about states, and it conditions on a variable measured after treatment whose incidence differs by arm (13/30 upper-state blocks for A, 7/30 for B, 14/30 for C). It is reproducible: `analyze_controls_posthoc.py` in the bundle states the threshold rule — cut at the single widest relative gap in the sorted process means, which is 2.11% against a next-widest gap of 0.25% — and prints every conditioned figure quoted here alongside its pre-registered marginal counterpart.

5.3.2 What the rejected implementation bought. The rejected heavyweight implementation retires fewer instructions than the candidate on all three count endpoints, unfavourably in every one of the 30 blocks, with Bonferroni-adjusted upper bounds of 1.00453, 1.00330 and 1.00329 against the pre-registered 1.01 noninferiority margin. Its extra saving is 11.0, 9.0 and 10.0 instructions per *stage iteration* — flat, where the candidate's saving is per counted document — out of the 116.1 to 134.1 that either implementation removes. That normalisation is the same test applied above to the candidate, and here it points the other way, which fits the mechanism: what B still does that C does not is replace the two virtual calls per iteration between `CountStage` and `CountScan` with typed ones.

That advantage does not survive contact with the un-optimized control. On the count whose plan cannot use

COUNT_SCAN, the rejected implementation retires **24 more instructions per fetched document** than either the candidate or the base, in both states (B/A is 1.002558 over 14 concordant blocks in the lower state and 1.002499 over 4 in the upper; B/C is 1.002564 over 11 and 1.002521 over 2), while the candidate is indistinguishable from the base there. This is consistent with the cost of routing every classic stage's `work()` through a new base-class helper, at roughly 8 instructions across the three `work()` calls per document. That is the largest of the B-only changes that execute on this plan — arm B also adds a null-pointer branch to `CountStage::doWork()` and `isEOF()` — but no arm isolates any of them, so the attribution is an inference and not a measurement. Note also that the pre-registered marginal estimator reports 0.9979 for this comparison and so inverts its sign: the rejected arm landed in the high-instruction state 7 times in 30 against the base's 13, and the mean over blocks absorbs the difference. The conditioned figure is reported alongside the pre-registered one rather than in place of it.

5.4 Validation and limits

The optimized build passes for `mongod`, `mongos`, `mongo` and `dbtest`; the query benchmarks were built separately, from the campaign's own worktree. Both count dbtest suites pass in optimized and runtime-`dassert` builds. Forced-classic resmoke passes all 60 result entries it produces: 42 over eight core files, 12 over two aggregation files, 3 over one no-passthrough file and 3 over one sharding file. Twelve of the 60 are the jstests themselves; the rest are their per-test `RunQueryStats` and `ValidateCollections` hooks and fixture setup and teardown. The core set includes `profile_count.js`, which checks indexed-count `keysExamined` and the `COUNT_SCAN` plan summary directly.

The strongest non-intrusion evidence is a field-by-field comparison of `explain("executionStats")` against a separately built binary of the base commit: over 542 leaf fields there are 4 differences and none is substantive, all four being the same optimization-time counter, three of whose four instances also differ between two runs of the identical binary. The comparison includes the count-like aggregation shape, which executes a bare `CountScan` with no `CountStage` and therefore dereferences the returned identifier unconditionally; it returns 200 materialized results on both binaries, so the opt-in does not leak to that path.

These validation runs are retained in the bundle under `validation/`, with the resmoke reports and logs, the dbtest logs, all three explain captures, the diffs and a hashed provenance record. They were performed on `ac20554f`, not on arm C: the two commits differ by exactly one line, a `clang-format` include reorder in `count.cpp`, and the validation was not re-run after it. No statement, declaration or build setting differs between them.

The result is limited to warmed, single-threaded, in-process counts on one host, and to plans forced by an index hint to the shape under test. It does not establish improvement for MongoDB 7.0.34, for SBE, for unhinted counts subject to multi-planning or the plan cache, for cold caches, for concurrent or mixed load, for remote clients, for the ConDB endpoint, or for production latency. CPU time here is benchmark-thread time, not whole-server or whole-process CPU. Retired instructions are not latency, and the reduction is stated as such. The two builds of arm C bracket the builds of arms A and B in one worktree and one build output base; the first was a build-cache hit and the second re-executed 1001 actions and reproduced the same binary, so what this evidences is that building the other two arms in between left the build state undisturbed. Both preceded the campaign, and it is not a from-scratch reproducibility proof.

Upstream submission remains blocked on matters that cannot be satisfied here: a real MongoDB ticket, the contributor-agreement and code-owner path, and an Evergreen run. The fork pull request is therefore a draft and is not represented as ready to merge. The complete protocol, source patches, per-arm manifests, build attestation, attempt ledger, raw JSON, logs and analysis are frozen under `bench/db/report/evidence/mongodb_master_countscan_20260805_4109dcc31ff6`.

Three disclosures about that bundle. First, the protocol was committed and pushed 40 seconds before the first benchmark process, but it was not written blind: the attested build smokes had already run all five workloads at campaign size on all three arms, so the per-arm instruction levels — and with them the endpoint ratios — were visible before the protocol was frozen, and recomputing those ratios from the smoke files reproduces the campaign result to within 4×10^{-4}. The attempt ledger discloses this in each of its three pre-registrations, but the frozen `campaign.json` does not: its noninferiority-margin rationale states that no measurement of the present arms had been taken, which is false. The protocol is left unaltered and the error is recorded here. What the 30-block campaign establishes is the intervals, the block-level consistency and the controls, not the discovery.

Second, the pre-registration cites the wrong anchor commit: it was written before its own commit existed and so recorded the then-current `1c1d4287`, which holds the previous attempt's protocol. The executed protocol first appears in `1688ea3d`. The freeze did precede execution; only the citation was wrong. Both the correction and the record of how it was filed are in `anchor_correction.json`.

Third, the campaign measured `90814b83d3e5`. A later blank review of the change, conducted as a MongoDB reviewer would read it, produced fixes — including an assertion that closes a second, previously unremarked `CountStage` construction site in `ClassicPlannerInterface::makeExecutor()` — and the commit now under review is `90781b36b2`. None of those fixes executes inside the per-key path, and the reviewed binary was re-smoked at campaign size. That smoke is one process per endpoint against a 30-process mean, not a repeat of the campaign: its instruction counts differ

from arm C's means by less than the range arm C's own 30 processes span, which on the multikey endpoint is about 2.3 of arm C's per-process standard deviations, or +0.54 instructions per counted document against an effect of 116. The complete diff, the smoke output and that comparison are in `validation/review_fix_equivalence/`. The intervals in Table 7 remain pinned to the measured commit.

6 OPTIMIZATION EXPERIMENTS

The 7.0.34 storage-harness profiles suggest two different interventions. Short reads can share fixed command setup by coalescing already-known inputs. Subtree reads may need a larger storage record rather than a different network transfer chunk. We test both ideas in that storage harness. Here, *validated* means output-equivalent within that harness; it does not mean integrated into ConDB's public API. None of the MongoDB interventions in this section is currently a production path.

The primary short-read sweep disables PyMongo command monitoring and uses 80 random input groups, five cyclically interleaved repeats, and two grouping seeds. Its 19,200 timed baseline–candidate pairs pass exact output-equivalence checks; repeated groups and IDs are not independent samples. A separate monitored run records command counts and produces nearly the same wall-time curve. Thus the reported wall time does not include one listener callback per database command. Table 8 gives representative endpoints. A separate three-arm control at $B \in \{1, 3, 8, 64\}$ uses 40 groups, three repeats, two grouping seeds, and at most 16 workers to compare sequential individual queries, concurrently submitted individual queries, and one `$in` query; even its 64-input arm never exceeds 16 client threads.

Table 8: MongoDB 7.0.34 storage-harness intervention endpoints. Short-read latency is wall time for one group of 64 already-known inputs under a sequential individual-query baseline. Subtree rows use separate paired cohorts and projections. The last column is the ratio of baseline to candidate latency at the marginal P50 and P95; it is not a percentile of per-input speedups. Compare only within a row.

Operation / intervention	Baseline P50 / P95 (ms)	Candidate P50 / P95 (ms)	Ratio at P50 / P95
`get_node`, $B = 64$	15.104 / 15.448	0.628 / 0.652	24.06 / 23.68×
`get_children`, $B = 64$	19.279 / 19.972	2.831 / 2.949	6.81 / 6.77×
`get_subtree`, ID-only 1M cursor request	10.671 / 118.552	10.155 / 115.188	1.05 / 1.03×
`get_subtree`, 256-row bucket	23.011 / 248.597	14.665 / 144.292	1.57 / 1.72×
`get_entity`, $B = 64$	13.382 / 13.639	0.533 / 0.559	25.10 / 24.41×

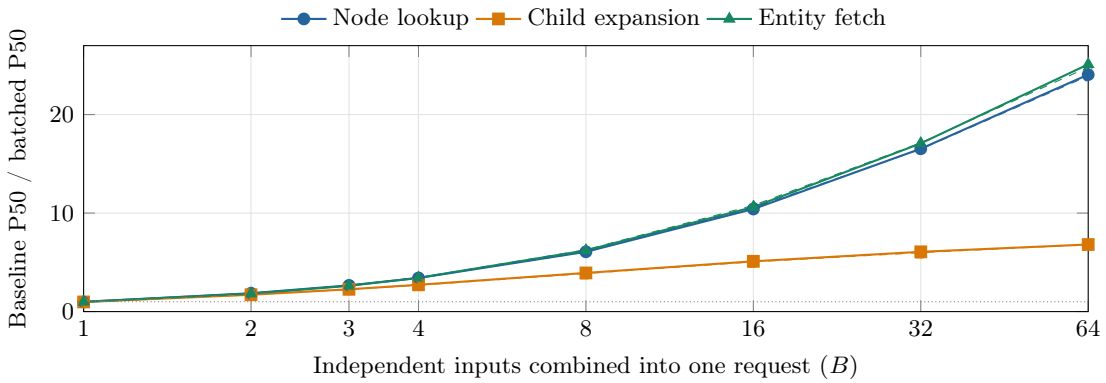


Figure 2: Wall-only batch-size sweep for the three short reads (80 groups, five repeats). The ratio compares B sequential equality queries with one output-equivalent `$in` query. Solid and dashed lines use two input grouping seeds in adjacent runs. The monotone trend, including the realistic $B = 3$ point, matches the expected effect of sharing fixed command setup. These are group-completion times, not concurrent-load throughput.

6.1 Node lookup

6.1.1 Optimization method. For each $B \in \{1, 2, 3, 4, 8, 16, 32, 64\}$, the baseline issues B equality `find` commands sequentially for a group of node IDs that is already known. The candidate sends the same IDs in one `$in` query, retains the `tree_id,node_id` index and projection, builds a map keyed by `node_id`, and restores caller input order. Thus index choice, returned records, and caller-visible order are held constant; the controlled change is B commands versus one multi-key command. The three-arm control submits those same B equality commands concurrently through the shared thread pool, preserving their query shape and output contract.

6.1.2 Experimental results. Node coalescing scales smoothly with group size. $B = 1$ is unchanged, $B = 3$ improves P50 from 0.729 to 0.274 ms (2.66×), and the ratio grows to 24.1× at $B = 64$. The second grouping seed stays within 0.4% at $B = 3$ and 0.6% at $B = 64$. In the separate telemetry run, command count falls from B to one and the wall-time curve remains close to the listener-free result. The gain comes from sharing command and query setup, not from a faster point lookup. With the 16-worker cap, `$in` remains 2.05×/1.79× faster than concurrently submitted individual queries at $B = 3$ across the two seeds, and 17.9×/18.3× faster at $B = 64$.

This helps only when the caller already has a group of independent IDs. It does not speed up a single lookup, and the benchmark does not include time spent forming a group. Its end-to-end value will depend on real group sizes and API-level load tests.

6.2 Child expansion

6.2.1 Optimization method. The baseline performs one indexed, ordered scan sequentially for each parent ID. The candidate uses one `parent_id:{$in:...}` query, sorts by `parent_id,path,node_id`, groups rows by parent, and emits each parent's children in the original path/node order. Both arms project the same child Metadata and use the same parent/path index. The candidate additionally pays for cross-parent ordering and client regrouping. Its threaded control submits the same per-parent scans concurrently through the shared pool without changing their individual query shape.

6.2.2 Experimental results. Returned-row work limits child coalescing. At $B = 3$, P50 improves 2.27×, but by $B = 64$ the ratio is 6.81×, far below the 24.1× point-read result. An average 64-parent group still returns about 643 child documents, so removing 63 command setups leaves scanning, document production, cross-parent ordering, regrouping, and output materialization. The curve begins to flatten after $B = 16$ as those per-child costs take over. Against individual queries submitted through the 16-worker-capped pool, the two seeds retain 1.81×/1.56× at $B = 3$ and 4.96×/5.07× at $B = 64$. Bounded concurrency reduces neither the benefit of coalescing nor the returned-row ceiling.

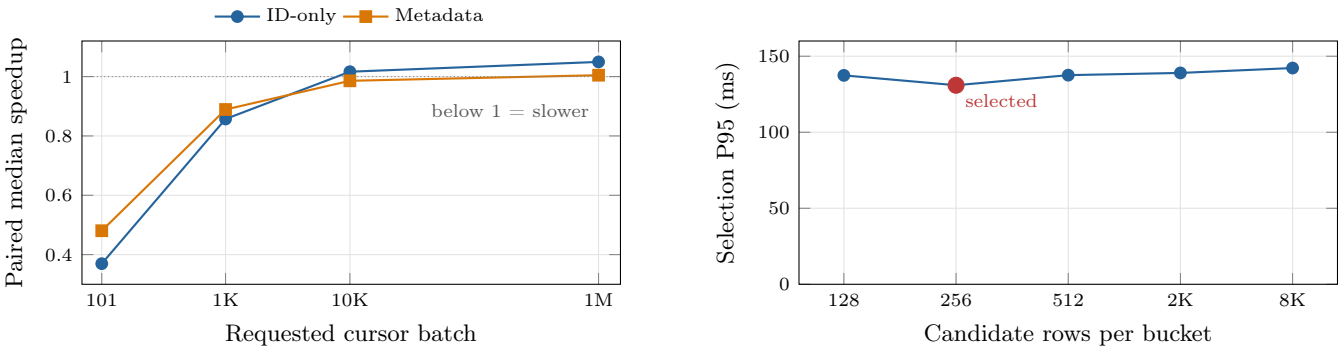
Coalescing remains useful when several parent IDs are already available, but it will not reach point-read-scale gains. Further improvement must reduce per-child fetch or output work.

6.3 Subtree retrieval

6.3.1 Optimization methods. Cursor batching. The first intervention changes only `Cursor.batch_size` for the same descendant-range query, index, projection, sort, and complete materialization. Both ID-only and covered-Metadata arms use the same 100 row-count-stratified ranges and five cyclically interleaved repeats; every variant and repeat stores an ordered-output digest. Primary wall-time runs register no command listener. A separate telemetry run records reply counts and durations. Requested sizes are 101, 1K, 10K, and 1M, but actual replies remain bounded by MongoDB message limits. The two projections test whether any cursor result survives a larger returned-record shape rather than treating ID-only behavior as universal. Root-bootstrap intervals resample ranges and are descriptive; they do not correct for shared hierarchy branches.

DFS buckets. The structural intervention packs consecutive DFS/path-order rows into parallel column arrays. A query performs the normal root lookup, two covered boundary probes, one contiguous bucket-sequence scan, and a range check on every decoded row; logically, only the two boundary buckets can contain out-of-range rows. This is a workload-specific use of MongoDB's Bucket Pattern.[3] Five candidate sizes (128, 256, 512, 2,048, and 8,192 rows) run together on 75 selection roots. The 256-row candidate has the lowest absolute selection P95; the winner is frozen before a separate runner measures 100 previously unqueried root IDs for seven paired repeats. Selection and holdout have no repeated root IDs, but they come from the same depth-1–3 large-subtree population on one tree and are not branch/range-disjoint: 19 selection–holdout ancestor relations affect 16 holdout roots. We report both the full frozen holdout and a conservative post-hoc sensitivity that removes every holdout root involved in any selection/holdout or within-holdout ancestor relation.

[3] MongoDB, "Group Data with the Bucket Pattern."



(a) Cursor-batch paired effects

(b) Bucket-size selection

Figure 3: The two subtree parameter sweeps. (a) Points are medians of per-range paired speedups and vertical bars are 95% root-bootstrap intervals. Small cursor batches are consistently worse. The modest ID-only effect at 10K/1M does not transfer to covered Metadata. (b) The 256-row bucket has the lowest selection P95 in this run, but the five candidates span only 8.6%; selection alone does not establish a generally optimal size.

6.3.2 Experimental results. Small cursor batches hurt both projections. At 101 and 1K rows, paired median speedups are 0.369 and 0.857 for ID-only output and 0.481 and 0.889 for covered Metadata. Telemetry shows why: the driver default uses a median of 2 commands, including 1 `getMore`, while 1K uses 12 commands and 101 uses 117.

Larger requests help only the narrow ID-only case. Its paired median reaches 1.017 at 10K and 1.049 at 1M; the 1M interval is $[1.038, 1.056]$, with marginal P50/P95 ratios of $1.05\times/1.03\times$. Covered Metadata stays at 0.986 and 1.005, with intervals $[0.970, 1.001]$ and $[0.998, 1.010]$. We therefore keep the driver default for the covered subtree path. Remote RTT, streaming, or early termination may have a different tradeoff.

The bucket sweep changes the storage record itself. The selection curve is shallow: 256 rows has the lowest P95, but all five sizes lie within 8.6%. An earlier run selected 2,048 rows and produced nearly the same holdout P50/P95 ratios ($1.57\times/1.73\times$ versus $1.57\times/1.72\times$). The bucketed layout matters more than the exact bucket size in this range.

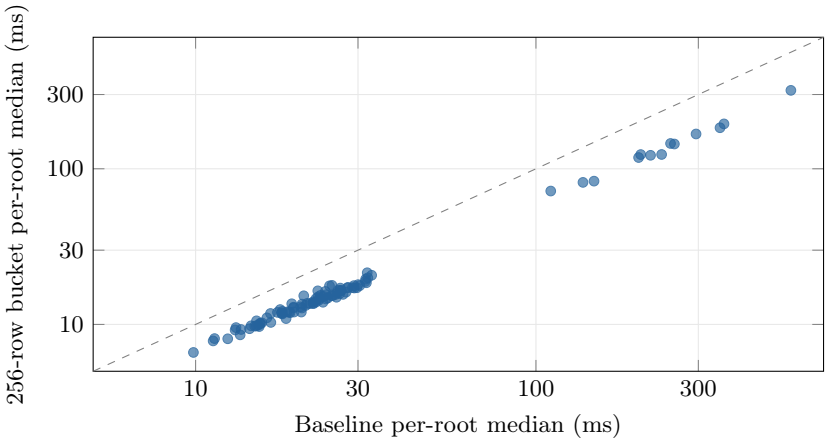


Figure 4: Paired DFS-bucket result for 100 previously unqueried, root-ID-disjoint inputs from the same large/shallow-subtree cohort. Coordinates are seven-repeat medians. The split is not branch/range-disjoint, so points are not 100 independent storage regions. All points fall below the equal-latency diagonal; logarithmic axes retain the cohort tail.

All 100 holdout roots improve. Marginal P50/P95 speedups are $1.57\times/1.72\times$, and 697 of 700 paired repeats are faster.

The split is root-ID-disjoint but not branch-disjoint. Removing every holdout root involved in a selection/holdout or within-holdout ancestor relation leaves 67 mutually range-disjoint roots. All 67 improve, with marginal P50/P95 speedups of $1.56\times/1.67\times$.

Fetch P95 falls from 172.5 to 71.2 ms because each BSON document carries many consecutive logical rows. The saving comes from fewer storage-engine record handoffs; boundary lookup, edge overfetch, array decoding, and client reconstruction remain.

Cost and validation. Rebuilding the 256-row layout produces 39,063 buckets in 77.0 s. A full pass rescans all 10M source rows, decodes the buckets, and reproduces the source digest with zero mismatches. The collection and its indexes occupy 1.450 GB, and the largest BSON document is 121.3 KiB. This adds 20.4% to the retained source collection and

its five indexes; against the 1.672 GB source collection alone, the increase is 86.7%. A hybrid route retains the existing structures, so the former denominator matches that design.

Buckets are a poor fit for small subtrees. Across 250 previously unused roots returning 265 rows on average, P50 rises from 1.044 to 2.171 ms and only 30 roots improve, although P95 falls from 3.769 to 3.348 ms. A serving path should not adopt size-based routing from these measurements alone: this study does not measure how subtree size is known before the read, the routing threshold and overhead, or false-route cost. Those controls are required before keeping small reads on the existing index and sending large reads to buckets. Mutable trees also need an update, rebuild, and atomic-publication design.

6.4 Entity fetch

6.4.1 Optimization method. For each batch size, the baseline issues sequential equality lookups on the Text collection's `_id`; the candidate sends one `_id:{$in:...}` query. Both arms explicitly use the same `_id` index, return the same `(_id,text)` fields, and restore caller input order with an in-memory ID map. The threaded control concurrently submits the same individual `_id` lookups through the shared pool.

6.4.2 Experimental results. Entity fetch follows the same curve as node lookup. A one-item batch is unchanged, $B = 3$ improves P50 from 0.698 to 0.267 ms (2.62×), and $B = 64$ reaches 25.1×; the second grouping seed stays within 2.0% at both points. Repeating the pattern in both the Nodes and Text collections points to shared command setup rather than a collection-specific effect. In the 16-worker-capped threaded control, the two seeds retain 2.28×/2.12× at $B = 3$ and 20.2×/19.1× at $B = 64$.

The main workload uses roughly 1 KiB texts. Larger entities encounter response-size and cursor-chunking costs, as the 256 KiB control in Section 4 shows. Multi-entity fetch is useful for naturally co-requested small texts.

6.5 Cross-engine implementation control (SQLite only)

6.5.1 Optimization method. This control tests whether the short-read direction survives in existing ConDB code; it is not a MongoDB implementation or a validation of MongoDB end-to-end latency. The SQLite storage layer now provides multi-parent-child and multi-entity reads. Inputs are deduplicated, missing entries remain explicit, every query is scoped by `tree_id`, and requests are split into 500-ID chunks. Child rows are regrouped by parent without changing slot order. Beam uses a batch method only when at least two uncached IDs are present; a one-ID read keeps the existing equality query. Nodes with a null `entity_id` do not issue an entity lookup. Storage implementations without the optional methods continue through the scalar path. Final contents bypass the cross-call entity cache, so batching does not weaken the previous fresh-read behavior.

The same change fixes a result-selection bug exposed by the first control. Previously, a ranked internal node was inserted into the final result even when the ranker returned `done=false`; with the default `select_k=1`, retrieval stopped before the next frontier. An internal node now advances to the next turn when `done=false`. A leaf or file is a final result, while an internal document node is final only when the ranker returns `done=true`, meaning that its content is already specific enough.

The latency control uses file-backed `TreeDB` copies and the real `BeamRetriever`. It keeps the public defaults `beam_size=3`, `select_k=1`, and the tree-depth turn limit. The tree has eight first-level sections and eight leaves per section; every section and leaf has a 1 KiB entity. A fixed ranker retains three sections on turn one with `done=false`, then selects one leaf on turn two. Both arms return exactly one leaf and one content object. The scalar arm hides the two batch methods; the candidate arm exposes them. SQL tracing is used only for an untimed count. Five rounds contain 250 paired repetitions per arm after 30 warmups, with arm order alternating within each round.

Table 9: File-backed Beam control with the public `beam_size` and `select_k` defaults. Times cover the complete deterministic two-turn retrieval. The ratio is scalar latency divided by batched latency.

Round	Scalar P50 / P95 (ms)	Batched P50 / P95 (ms)	Ratio P50 / P95
1	0.735 / 0.865	0.618 / 0.728	1.189 / 1.188×
2	0.732 / 0.861	0.615 / 0.726	1.191 / 1.186×
3	0.735 / 0.859	0.625 / 0.727	1.177 / 1.181×
4	0.726 / 0.807	0.610 / 0.697	1.191 / 1.157×
5	0.725 / 0.857	0.608 / 0.718	1.192 / 1.194×

6.5.2 Experimental results. The scalar arm performs one root-child read, eight first-turn entity reads, three frontier-child reads, 24 second-turn entity reads, and one final entity read. These 37 batchable reads become five. The shared root-ID and maximum-depth lookups bring the full counts to 39 and seven `SELECT`s. P50 improves by 1.177–1.192× across

the five rounds, a 15.1–16.1% latency reduction; P95 improves by 1.157–1.194×, a 13.5–16.3% reduction. The smaller wall-time gain is expected: prompt rendering, candidate construction, JSON decoding, and result materialization are unchanged.

A second control opens one SQLite connection per client and runs 100 queries per client at one, four, and 16 clients. Three rounds alternate arm order. The 16-client limit is enforced before a database is opened or a worker is created. Table 10 reports the median arm result across the three rounds.

Table 10: File-backed load control. Throughput and P50 latency are medians across three rounds. Ratios are the medians of the three within-round paired ratios: batched/scalar for throughput and scalar/batched for latency.

Clients	Scalar / batched q/s	Throughput ratio	Scalar / batched P50 (ms)	Latency ratio
1	1224.7 / 1501.4	1.207×	0.788 / 0.629	1.213×
4	443.3 / 551.3	1.239×	9.108 / 7.234	1.257×
16	237.3 / 272.2	1.157×	67.168 / 58.590	1.156×

Every paired round favors batching. At 16 clients, the medians of the paired throughput and P50 ratios are 1.157× and 1.156×. The separately aggregated arm medians correspond to 14.7% higher throughput and 12.8% lower P50 latency. Median arm P95 falls from 75.128 to 65.743 ms; the median paired P95 ratio is 1.147×. Neither arm scales with client count: contention and Python-side work drive throughput down and latency up. Coalescing reduces that cost but does not make the current SQLite path a high-concurrency design.

The raw results are stored in:

`bench/db/sqlite_beam_batching_20260729.json`
`bench/db/sqlite_beam_batching_load_20260729.json`

Each records its command line, Git head, output fingerprint, and source-file hashes. The public scalar navigation methods, other tree shapes, and mutable trees remain outside this control.

The companion file `bench/db/report/MONGODB_OPTIMIZATION_REVIEW.md` maps the MongoDB claims to their frozen artifacts and SHA-256 values, distinguishes component evidence from implemented behavior, and records the remaining adoption gates. The large raw measurement JSON files under `bench/db/runs` are not versioned by default; external reproduction therefore requires publishing that frozen evidence bundle in addition to this report and its manifest.

7 CONCLUSION

Request coalescing is now integrated into the SQLite-backed Beam path. Under the report's local, warm-cache, fully materialized conditions, the storage experiment shows that it helps with only three inputs and remains faster than submitting individual queries through a 16-worker pool. The cross-engine implementation control confirms the direction but also bounds the claim: reducing 37 batchable reads to five yields a repeatable 1.18× single-client gain and a 1.16× median paired throughput gain at 16 clients, not a statement-count-sized speedup. Under these same conditions, cursor batch tuning is not worth pursuing for the fully materialized covered-Metadata subtree path: small batches add `getMore` traffic, while large requests fail to improve Metadata retrieval. Remote RTT, streaming, and early termination remain unmeasured.

Separately, a direct `CountStage`→`CountScan` pair that skips an unread working-set member reduces benchmark-thread user-space retired instructions against a pinned upstream base carrying the same benchmark harness by 4.600%, 2.055% and 3.680% on the three indexed-count shapes measured, favourably in all 30 blocks and consistent at about 117 instructions per counted document. The optimization fires wherever `CountStage`'s child is a bare `CountScan`. `QueryPlanner::plan` returns a count scan as the single solution as soon as one is available, so that is every classic `COUNT_SCAN` plan rather than an occasional case; counts that plan to `FETCH` or a collection scan are untouched. Every plan measured is forced by an index hint. That campaign's pre-registered adoption gate did not pass: its point-query control showed a binary-level CPU-time offset that the change cannot cause, and its un-optimized control had a band that was never attainable at 30 blocks. No CPU-time or wall-time result is claimed. A post-hoc analysis of that second control, conditioning on a two-state instruction latch that the pre-registered estimator averages over, indicates that an earlier, rejected implementation of the idea intrudes on a plan shape where the optimization cannot fire; the pre-registered marginal estimator inverts the sign of that comparison, and both figures are reported in Section 5. This narrow source experiment is not a MongoDB version comparison or evidence of ConDB, whole-process, concurrent, or production improvement.

DFS buckets improve the measured large-subtree cohort but are not ready for an adoption decision because small reads regress and the size-based router itself is unmeasured. The next experiments should measure the same Beam path

on additional trees, decide whether public navigation needs batch methods, and evaluate the size estimator, routing threshold, and false-route cost on branch-disjoint trees with mixed subtree sizes. Mutable trees also need measured rebuild and publication behavior.